Qwen

От эму145 до Rust: Проектирование цикло-точного эмулятора Электроника МК-52 для глубокого изучения архитектуры микроконтроллеров

Архитектурный Синтез: Деконструкция Уникальной Многопроцессорной Системы МК-52

Исследование и последующая разработка точного эмулятора Электроника МК-52 требуют фундаментального понимания его уникальной аппаратной архитектуры. В отличие от более простых одночиповых микроконтроллеров, таких как Intel 8080 или классические игровые консоли, МК-52 представляет собой сложную систему, построенную на основе нескольких специализированных микросхем, работающих в тесном взаимодействии [11] [20]. Этот факт является ключевым фактором, определяющим как саму природу задачи эмуляции, так и предпочтительную архитектуру программного решения. Проект не сводится к простой эмуляции одного центрального процессора; он требует моделирования целой "мозговой группы" из нескольких микроконтроллеров, разделенных между собой функциональные обязанности и общающихся через общую структуру данных. Такой подход позволяет не только воспроизвести базовые вычислительные операции, но и детально изучить принципы распределенной обработки информации и синхронизации компонентов, что полностью соответствует образовательным целям проекта. Анализ предоставленных источников позволяет провести деконструкцию этой сложной системы, выявив ее основные строительные блоки, механизмы взаимодействия и особенности организации памяти.

Центральным элементом архитектуры МК-52 является не один монолитный процессор, а несколько чипов, каждый из которых выполняет свою уникальную роль [20]. Эти компоненты работают параллельно, но синхронно, формируя единый аппаратный комплекс. Основные микросхемы, составляющие ядро системы, включают: К145ИК1302-2 (или К745ИК1302-2), который выступает в

роли диспетчера и главного контроллера ввода-вывода; К145ИК1303-2, отвечающий за выполнение арифметических операций; К145ИК1306-2, предназначенный для дополнительных функций и логических операций над регистрами; а также К745ИК1801-2, который служит микроконтроллером для управления интерфейсом EEPROM и основной памятью [11] [20]. Кроме того, система включает в себя КР1601РР1 — чип внешней электронно-писемой постоянной памяти (EEPROM) емкостью 512 байт, и К745ГФ3-2 — четырехфазный генератор тактовых импульсов, обеспечивающий синхронизацию всех процессов [1] [11]. Эта многоядерная концепция, где каждая часть оборудования имеет свой собственный "мозг", является отправной точкой для проектирования эмулятора. Любая попытка смоделировать работу устройства путем создания одного абстрактного процессора потерпит неудачу, поскольку ключевые аспекты поведения, такие как обработка команд и взаимодействие с периферией, происходят именно на уровне этих специализированных микроконтроллеров. Эмулятор должен быть спроектирован как набор взаимосвязанных объектов, каждый из которых будет представлять один из этих чипов, их состояние и логику работы.

Ключевым элементом, объединяющим все эти микроконтроллеры, является общая структура данных — круговой буфер, организованный на базе шифраторов [20]. Вся информация, необходимая для работы калькулятора, хранится в этом буфере: значения пользовательских регистров, данные стека, текущий программный код и даже сами микрокоманды, из которых состоит каждый машинный язык. Общий объем этого буфера составляет 2352 бита [20]. Это означает, что эмуляция должна выходить за рамки простого моделирования регистров и памяти; необходимо воссоздать всю эту сложную структуру и механизм доступа к ней. Данные в буфере хранятся нестрого, а в определенном порядке: для представления чисел используется метод, при котором полубайты числа располагаются в каждом третьем бите буфера, а для доступа к данным происходит последовательный обход областей M1 (регистры), M2 (стек) и M3 (программа) [20]. Этот нетривиальный способ организации памяти напрямую связан с особенностями самого процессора. ALU (арифметико-логическое устройство) в основе МК-52 является однобитным и работает с одним полубайтом за микроцикл, что требует четкой синхронизации доступа к данным в буфере [20]. Таким образом, эмулятор должен не просто предоставлять массив памяти, а точно моделировать этот сложный механизм перемещения данных и их обработки в рамках четырехфазного цикла.

Синхронизация всей системы осуществляется с помощью четырехфазного тактового сигнала, генерируемого чипом К745ГФ3-2 [1] [11]. Частота системы составляет примерно 75 кГц [3] [4]. Каждый тактовый цикл делится на четыре микроцикла, в течение каждого из которых каждый из микроконтроллеров выполняет одну микрооперацию [20]. Эта последовательность обеспечивает правильную передачу данных и координацию действий между процессорами. Например, после выполнения операции одним процессором результат записывается в специальную область буфера, называемую "Каналом Связи" (КС), где другие чипы его ожидают [20]. Главный процессор К145ИК1302 помещает в этот канал код команды, другой процессор (например, К145ИК1303) его считывает, выполняет требуемое действие и возвращает код результата (например, F1 для успеха или F0 для ошибки) [20]. Для этого используется механизм тайм-аута, чтобы определить, ответил ли другой процессор на команду [20]. Эта модель взаимодействия "запрос-ответ" с обязательной задержкой, соответствующей времени выполнения команды, является еще одной причиной, по которой простая инструкционная эмуляция недостаточна. Для достижения полной точности необходимо моделировать каждый шаг, каждый микроцикл, и время, затрачиваемое на обмен сообщениями между модулями.

Для полного понимания архитектуры важно рассмотреть и другие компоненты системы, которые не являются частью основного процессингового ядра, но играют важную роль в общем функционировании. Одним из них является EEPROM-модуль, представленный чипом КР1601РР1, который обеспечивает неизбыточное хранение программ и данных [1] [11]. Этот чип способен выдержать до 10 000 циклов записи и может хранить до 512 байт, что эквивалентно 72 регистрам по 7 байт каждый или 105 шагам программы [1] [11]. Управление этим чипом осуществляется отдельным микроконтроллером К745ИК1801-2 [11]. Программирование EEPROM требует высокого напряжения около -56В, которое генерируется специальным DC-DC преобразователем на отдельной плате [27]. Этот же преобразователь питает и сам дисплей. Еще одна важная плата содержит саму клавиатуру, которая соединена с основной платой гибким шлейфом [19]. Все эти периферийные устройства имеют свои собственные контроллеры или модули управления, что еще раз подчеркивает распределенную природу архитектуры МК-52.

Компонент	Модель	Назначение
Главный контроллер	К145ИК1302-2 / К745ИК1302-2 [11]	Диспетчеризация команд, управление вводом-выводом, сканирование клавиатуры, управление VFD [20]
Арифметический процессор	К145ИК1303-2 [11]	Выполнение математических операций (сложение, вычитание, тригонометрические функции и т.д.) [20]
Процессор дополнительных функций	К145ИК1306-2 [11]	Логические операции, работа с регистрами, дополнительные функции [20]
Интерфейс EEPROM/ Памяти	К745ИК1801-2 [11]	Управление внешней EEPROM (КР1601РР1) и общей памятью [11]
EEPROM	КР1601РР1 [1] [11]	Хранение программ и данных без питания (512 байт, 10 000 циклов записи) [1] [11]
Генератор тактовых сигналов	К745ГФ3-2 [1] [11]	Создание четырехфазного тактового сигнала для синхронизации всех компонентов (≈75 кГц) [1] [11]
Вакуумно-флюоресцентный дисплей	ИЛЦ2-12/8Л [1]	Индикация результатов (10 цифр + знак экспоненты) [1] [4]
Преобразователь напряжения	Специализированный модуль	Генерация высокого отрицательного напряжения (-27В, -56В) для VFD и EEPROM [4] [27]

Эта деконструкция архитектуры MK-52 показывает, что для создания полноценного учебного прототипа необходимо инициировать не один, а несколько уровней абстракции. На самом нижнем уровне находится модель кругового буфера, содержащего все данные системы. Над ним расположены модели отдельных микроконтроллеров (К145ИК13xx), каждый из которых реализует свою логику обработки данных и взаимодействия с буфером. На верхнем уровне находятся модели периферийных устройств, таких как контроллер VFD и сканер клавиатуры, которые также взаимодействуют с буфером и друг с другом через специальные механизмы синхронизации. Такая многоуровневая, модульная архитектура является наиболее адекватной для точного воспроизведения поведения оригинального устройства и, следовательно, для достижения поставленной образовательной цели. Она позволяет исследовать не только то, что делает процессор, но и когда он это делает в синхронии с остальной системой, что является ключевым для понимающего микроконтроллеров.

Парадигма Цикло-Точной Эмуляции: Обоснование и Практическая Реализация в Rust

Запрос пользователя на создание эмулятора с "полным воспроизведением оригинального поведения" является центральным и определяющим для всего проекта. Этот запрос требует перехода от стандартных подходов к эмуляции, таких как инструкционно-точная, к гораздо более сложной и точной парадигме — цикло-точной эмуляции. Понимание и реализация этой парадигмы является не просто желательным дополнением, а фундаментальным требованием для достижения поставленной образовательной цели, поскольку именно она позволяет воспроизвести тонкие временные зависимости и побочные эффекты, характерные для реального аппаратного обеспечения MK-52. Анализ предоставленных материалов показывает, что цикло-точная эмуляция не только необходима, но и практически реализуема с использованием современных возможностей языка Rust.

Цикло-точная эмуляция определяется как модель, в которой взаимодействия между аппаратными компонентами происходят с тем же временным интервалом, что и на оригинальном устройстве, что обеспечивает идентичное поведение машины для любого входного сигнала [8]. В системах с несколькими тактовыми частотами, как, например, NES, где графическая подсистема (PPU) работает быстрее центрального процессора (CPU), это означает, что эмулятор должен моделировать работу каждой подсистемы в своем собственном тактовом цикле, причем эти циклы должны проходить с тем же соотношением скоростей, что и в оригинале [9]. Для MK-52, где процессы ввода-вывода (сканирование клавиатуры, управление VFD) и выполнение программного кода происходят параллельно и тесно переплетены во времени, эта точность становится абсолютно критичной [5]. Если эмуляция не будет точно воспроизводить тайминги, то многие аспекты работы устройства, включая корректный ввод с клавиатуры и нормальное функционирование дисплея, будут работать некорректно или вовсе не работать. Простая инструкционно-точная эмуляция, где каждый шаг — это выполнение целой инструкции, приведет к значительным временным дисторсиям, так как она не учитывает длительность промежуточных операций, таких как чтение из памяти или обмен данными между процессорами [17].

Необходимость цикл-точной эмуляции для MK-52 подтверждается анализом его ключевых периферийных функций. Управление вакуумно-флюоресцентным

дисплеем (VFD) является ярким примером такого процесса. VFD работает по принципу мультиплексирования: для освещения одной цифры используется отдельный сеточный сигнал, который активируется на короткий промежуток времени (около 135 микросекунд) [5]. После этого экран гаснет, и следующий сеточный сигнал зажигает следующую цифру. Этот цикл повторяется последовательно. Критически важно, что между этими циклами освещения соседних цифр существует так называемый "интервал сканирования интервала" (ISG), в течение которого контроллер переходит к сканированию клавиатуры [5]. Таким образом, тайминг VFD и тайминг сканера клавиатуры жестко синхронизированы. Если эмулятор будет слишком быстро перебирать циклы, то он может пропустить окно для сканирования клавиатуры, и нажатия кнопок не будут зарегистрированы. Если же он будет медленнее оригинала, то клавиатурный ввод будет работать, но вся работа калькулятора будет казаться замедленной. Только цикл-точная эмуляция, где каждый микроцикл (соответствующий 75 кГц) моделируется последовательно, позволит точно воспроизвести эту синхронизацию и обеспечить корректную работу обоих подсистем одновременно.

Более того, цикл-точность является единственным способом воспроизвести известные недокументированные функции и "баги" MR-52, которые являются ценным учебным материалом. Например, состояние "EGGOG", являющееся формой вывода ошибки, может возникать не только при истинных ошибках, но и при арифметических переполнениях, и его возникновение зависит от точного момента, когда результат вычисления становится невалидным [3]. Состояние "Тьма" ("Darkness"), при котором экран полностью гаснет, достигается выполнением очень специфической последовательности операций, зависящей от состояния дисплея, которое изменяется с каждым циклом [3]. Наиболее интересный пример — "пустышка", пользовательский баг, вызываемый случайным появлением в памяти уникального 9-полубитного паттерна, состоящего из одних единиц [20]. Обнаружив этот паттерн, внутренний синхронизатор системы сбрасывает счетчик, что приводит к полной потере синхронизации и коррупции памяти [20]. Воспроизведение такого бага возможно только при точном моделировании каждого шага, при котором паттерн "пробегает" по памяти. Инструкционно-точная эмуляция, пропускающая промежуточные состояния, никогда не сможет запустить это событие. Таким образом, выбор цикл-точной эмуляции напрямую соответствует цели изучения не только документированной, но и "темной стороны" архитектуры микроконтроллеров.

Практическая реализация цикл-точной эмуляции в Rust является вполне осуществимой задачей, хотя и требует продуманного подхода к архитектуре. Источники демонстрируют несколько успешных примеров цикл-точных эмуляторов, написанных на Rust, таких как x-nes (NES) и nes-emu, что служит хорошим практическим ориентиром [14] [15]. Один из наиболее интересных подходов, описанный в контексте эмуляции NES, заключается в использовании Rust-корутины (функция-генератора с `yield`), которые пока что являются нестабильной функцией [17]. Преобразование логики CPU в корутину позволяет писать код в виде удобного цикла инструкций, при этом явно указывая точки, где нужно "приостановиться" на один тактовый цикл (`yield`). Компилятор автоматически преобразует это в эффективную конечную автомату, что значительно упрощает реализацию шагового прогресса по циклам по сравнению с ручным управлением состоянием [17]. Хотя использование нестабильных фич может быть ограничением для некоторых проектов, для научно-исследовательской цели данного проекта это может быть приемлемым компромиссом ради максимальной точности и читаемости кода.

Независимо от использования корутин, ключевой принцип архитектуры — это разделение на бэкенд (логика эмуляции) и фронтенд (графический интерфейс). Эта модульная структура, часто используемая в эмуляции, позволяет отделить сложную внутреннюю логику от платформо-зависимого вывода [16]. В данном случае бэкенд будет представлять собой набор взаимосвязанных объектов, моделирующих каждый из микроконтроллеров МК-52 (К145ИК1302, К145ИК1303 и т.д.), контроллер EEPROM и другие компоненты [13]. Каждый такой объект будет иметь метод `tick()`, который будет моделировать один тактовый цикл своей работы. Основной цикл эмулятора будет последовательно вызывать этот метод для каждого компонента, либо сдвигаясь на несколько тактов для CPU, если другие компоненты (например, PPU) работают быстрее, как это принято в эмуляции NES [17]. Взаимодействие между компонентами будет осуществляться через общий объект памяти (круговой буфер), который они будут общей. Для передачи данных и команд между компонентами можно использовать каналы сообщений, аналогично тому, как это сделано в эмуляторе YM2149, где эмулятор звука взаимодействует с CPU через API для записи регистров [13].

Фронтенд, в свою очередь, будет построен с использованием библиотеки `egui` поверх `eframe`, что идеально подходит для создания быстрого и легковесного десктопного приложения [6]. `egui` является библиотекой немедленного режима, что означает, что она не сохраняет состояние UI-элементов, а генерирует их

заново каждый кадр на основе текущего состояния приложения [6] [7]. Это хорошо сочетается с задачей отображения состояния эмулятора, которое меняется часто. Основной цикл фронтенда будет заниматься сбором ввода (клавиатура, мышь) и отрисовкой UI, в то время как бэкенд будет работать в фоновом потоке, выполняя циклы эмуляции. Для обмена информацией между потоками можно использовать стандартные средства Rust, такие как каналы `std::sync::mpsc::channel` [6]. Например, после выполнения достаточного количества тактов, чтобы обновить состояние VFD, бэкенд будет отправлять в канал структуру с новым изображением дисплея. Фронтенд получит это сообщение и отобразит его в окне `egui`. Аналогично, события нажатия кнопок на экранной клавиатуре будут отправляться в бэкенд, который их обработает как будто бы была нажата физическая кнопка. Такой подход обеспечивает чистое разделение ответственности, высокую производительность (можно реализовать паузу эмуляции при потере фокуса окна) и гибкость, позволяя легко менять фронтенд в будущем, сохранив при этом весь сложный бэкенд [16].

Ресурсный Ландшафт: Анализ Предшествующих Эмуляторов и Исходных Кодов

Одним из наиболее значительных преимуществ при разработке нового эмулятора MK-52 является наличие богатого ресурсного ландшафта, который включает как исчерпывающую документацию, так и существующие программы-эмуляторы с открытым исходным кодом. Этот факт кардинально снижает первоначальный порог входа и позволяет сосредоточиться не на фундаментальном исследовании архитектуры, а на ее точной реализации в среде Rust. Ключевым элементом этого ландшафта является проект `emu145` от Felix Lazarev, который стал своего рода "золотым стандартом" для эмуляции всей серии микроконтроллеров 145/745. Анализ этого и других предшествующих работ позволяет сформировать четкую стратегию для данного проекта, основанную на реинжиниринге проверенной годами технологии.

Проект `emu145` представляет собой не просто эмулятор, а результат глубокого анализа и восстановления содержимого ROM-чипов серии IK13xx, которые являются сердцем калькуляторов MK-61 и MK-52 [28] [29]. Автор проекта, Felix Lazarev, в сотрудничестве с Eugene Trinity, смог получить фотографии

кристаллов этих чипов и декодировать их микрокод [28]. Это было огромным прорывом, поскольку до этого момента многие аспекты работы калькуляторов, включая некоторые неофициальные возможности, оставались лишь догадками [29]. Благодаря этому проекту стали доступны точные инструкции, из которых состоят все машинные команды, что позволило создавать эмуляторы, полностью совпадающие с оригинальным поведением, включая неточности вычислений и побочные эффекты [24] [26]. Именно поэтому многие современные эмуляторы, такие как Java-версия от Vadim Khokhlov и Android-приложение "МК 61/54", используют исходный код emu145 в качестве основы [22] [24] [26]. Для проекта по созданию эмулятора на Rust это означает, что самый эффективный и надежный путь — это не писать эмулятор с нуля, а взять существующий C++ код emu145 и перевести его логику на Rust.

Стратегия реинжиниринга emu145 предполагает несколько этапов. Во-первых, необходимо тщательно изучить исходный код C++, понять его архитектуру, структуру данных и логику работы каждого компонента. emu145 уже содержит точную реализацию моделирования микрокоманд, что является основой для цикл-точной эмуляции. Во-вторых, следует спроектировать аналогичную архитектуру на Rust, создав структуры и трейты, которые будут соответствовать компонентам оригинальной системы. В-третьих, начинается процесс пошагового перевода логики из C++ в Rust, адаптируя ее к современным практикам языка (например, владение данными, безопасность типов) и добавляя необходимые элементы для цикл-точной модели. При этом важно не просто скопировать код, а перестроить его в соответствии с требованиями цикл-точности, которую предлагает emu145. Это позволит сразу получить доступ к проверенной годами точной модели работы микроконтроллеров, что является краеугольным камнем всего проекта.

Помимо emu145, существует ряд других полезных ресурсов. Сканы официальных руководств по эксплуатации и программированию МК-61 и МК-52 служат отличной отправной точкой для понимания пользовательского интерфейса, системы команд и Reverse Polish Notation (RPN), которую использует калькулятор [29] [30]. Эти документы описывают все официально заявленные функции, что помогает в тестировании эмулятора. Программные коллекции, такие как те, что собраны на сайтах XVadim.XYZ, предоставляют готовый набор тестов для проверки точности эмуляции [22]. В эту коллекцию входят знаменитые игры, такие как "Лунолет-2", а также различные утилиты и программы для решения математических задач, что позволяет проверить работу эмулятора в самых разных сценариях [22] [31]. Также имеются статьи на

русскоязычных ресурсах, например, на Habr, которые дают хороший обзор архитектуры, хотя и могут содержать менее глубокие технические детали по сравнению с другими источниками [3] [10].

Ресурс	Формат	Описание	Значимость для проекта
emu145 project by Felix Lazarev	Исходный код (C++)	Проект, восстановивший микрокод ROM-чипов серии 145/745. Является основой для многих точных эмуляторов.	Критически важный. Основа для реинжиниринга. Обеспечивает высочайшую точность эмуляции.
Extended MK-61/54 Emulator	Исходный код (Java)	Эмулятор, основанный на emu145, с оптимизациями и дополнительными функциями (сохранение состояний).	Полезен для понимания архитектуры и возможных доработок emu145.
Android "МК 61/54" app	Исходный код (Java)	Мобильное приложение, использующее эмулятор на базе emu145.	Подтверждает актуальность и востребованность точной эмуляции.
Official manuals (МК-61/МК-52)	PDF (Сканы)	Официальная документация по программированию и эксплуатации.	Необходима для понимания пользовательского интерфейса и тестирования базовых функций.
Collection of programs for MK-61/54	Архив программ	Коллекция программ, включая игры и утилиты, для тестирования эмулятора.	Критически важный. Предоставляет набор реальных нагрузок для проверки точности.
Habr articles	Статьи	Хабр-статьи, обобщающие архитектуру и историю советских калькуляторов.	Хорошее введение, но может не содержать всей глубины технических деталей.
Hardware repair guides	Статьи	Статьи, описывающие ремонт аппаратных модулей (VFD, клавиатура, блок питания).	Полезно для точной симуляции работы периферии и диагностики проблем.

Для обеспечения максимальной точности и полноты воспроизведения, проект должен быть сфокусирован на воссоздании не только официальных, но и неофициальных характеристик МК-52. Как упоминалось ранее, в этих калькуляторах есть множество "темных входов" и побочных эффектов, которые были широко известны среди энтузиастов [2] [11]. Например, возможность ввода шестнадцатеричных чисел с использованием буквенных клавиш, которые по сути являются кириллическими буквами (А=0, В=L, С=С и т.д.), и последующая обработка этих чисел приводит к интересным результатам, таким как отображение "Err0r" [12]. Также существуют способы хранения текстовых строк в регистрах с помощью комбинации арифметических и логических функций [12]. Эти особенности являются не ошибками, а следствием уникальной архитектуры и должны быть точно воспроизведены в эмуляторе, чтобы он был по-настоящему полным учебным прототипом. Проект `mk-pro`, который включает в себя JavaScript-эмулятор в качестве авторитетной модели для проверки оптимизаций, демонстрирует важность учета таких "темных входов" [32]. Следовательно, при переводе `emu145` на Rust необходимо уделять особое

внимание воспроизведению всех этих нестандартных поведений, которые заложены в микрокоде.

Воспроизведение Периферии: Симуляция VFD и Клавиатуры с помощью egui

Одной из ключевых задач проекта является создание точной симуляции пользовательского интерфейса калькулятора Электроника МК-52 с использованием библиотеки `egui`. Поскольку оригинальное устройство оснащалось вакуумно-флюоресцентным дисплеем (ВФД) и механической клавиатурой, а не графическим ЖК-экраном, требуется разработать программные моделировщики этих двух компонентов. Успешная реализация этой задачи не только обеспечит визуальное соответствие оригиналу, но и станет важным аспектом для достижения цели цикл-точной эмуляции, поскольку работа дисплея и сканера клавиатуры жестко синхронизирована с основным процессором. Выбор `egui` в качестве инструмента для визуализации является стратегически верным, так как эта библиотека для немедленного режима предоставляет гибкость и скорость разработки, идеально подходящие для создания интерфейсов эмуляторов [6].

Моделирование VFD является нетривиальной задачей, требующей понимания его физической природы и принципа работы. VFD в МК-52 — это высоковольтный дисплей, требующий для работы напряжения около -27В [4] [5]. Он состоит из десяти цифровых индикаторов (8 для мантиссы и 2 для экспоненты) и нескольких специальных символов [1] [4]. Принцип его работы основан на мультиплексировании: в любой момент времени освещается только одна цифра [5]. Контроллер устанавливает нужные сегменты для конкретной цифры, а затем подает на соответствующий "сеточный" электрод короткий импульс (около 135 микросекунд), в результате чего эта цифра светится. Затем контроллер переходит к следующей цифре, и так далее по цепочке [5]. БлагодаряPersistence of vision эффекту, человеческий глаз видит все десять цифр как постоянно горящие. Этот процесс должен быть точно смоделирован в эмуляторе. Внутри бэкенда эмулятора необходимо создать структуру данных, представляющую десять сегментов дисплея. В каждом тактовом цикле эмулятор должен определять, какая цифра сейчас должна быть освещена, и обновлять состояние этой цифры в своей внутренней модели дисплея. После

завершения цикла освещения одной цифры (что соответствует определенному количеству тактов) эмулятор должен переключиться на следующую. Эта синхронизация является ключевой для корректной работы клавиатурного ввода.

Связь между VFD и клавиатурой еще более тесная. Как было установлено, между двумя циклами сканирования VFD вставляется "пробел для сканирования интервала" (ISG), в течение которого контроллер переходит к сканированию клавиатуры [5]. Это означает, что эмуляция должна не просто отрисовывать VFD, но и в нужные моменты времени "подсвечивать" клавишу, которая была нажата. Для этого в модели эмулятора должна быть предусмотрена возможность получения сигналов о нажатии клавиш из фронтенда (от `egui`) и их обработки в том же цикле, когда контроллер сканирует клавиатуру. Когда эмулятор определяет, что в данный момент времени происходит сканирование клавиатуры, он проверяет, какая из клавиш была нажата в фронтенде, и сообщает об этом соответствующему процессору. Это обеспечивает полную синхронизацию между визуальным отображением дисплея и реактивностью на ввод. Кроме того, необходимо смоделировать и другие особенности работы VFD. Например, во время выполнения долгих вычислений экран может на короткое время гаснуть, а промежуточные значения во время исполнения программы могут мелькать на экране [5]. Эти визуальные эффекты, хотя и кажутся второстепенными, являются важной частью "характера" оригинального устройства и их точное воспроизведение повысит уровень достоверности эмуляции.

Характеристика	Оригинал (MK-52)	Эмуляция в Rust (egui)
Тип дисплея	Вакуумно-флюоресцентный (VFD) [4]	Программная модель 7-сегментных индикаторов
Разрешение	10 цифр (8+2) + специальные символы [1] [4]	Массив из 10 структур, описывающих состояние каждого сегмента
Принцип работы	Мультиплексирование, сканирование по очереди [5]	Цикл эмулятора, переключающий отображаемую цифру кажд N тактов
Высокое напряжение	~ -27V [4] [5]	Не моделируется (логический уровень)
Сканирование клавиатуры	Через "пробел для сканирования интервала" (ISG) [5]	Проверка состояния клавиатуры в фронтенде, передача в бэкенд в нужный тактовый цикл
Визуальные эффекты	Гашение экрана при вычислениях, мелькание промежуточных значений [5]	Управление видимостью дисплея в бэкенде на основе состояния процессора
Интерфейс	Механическая клавиатура [19]	Виртуальная клавиатура на основе `egui`

Архитектура взаимодействия между бэкендом и фронтендом `egui` должна быть продумана для эффективной передачи данных и минимизации задержек. Как уже упоминалось, бэкенд должен работать в отдельном потоке, выполняя циклы эмуляции. Он будет хранить в защищенных структурах (`Arc<Mutex<...>>` или каналы) текущее состояние дисплея и другие параметры. Фронтенд, работающий в основном потоке, будет в каждом кадре запрашивать у бэкенда обновленное состояние дисплея. Для отрисовки VFD в `egui` можно использовать несколько подходов. Самый простой — создать изображение (текстуру) 10 отдельных 7-сегментных цифр и в каждом кадре рисовать только те сегменты, которые должны быть зажжены. Более сложный, но и более точный подход — использовать `egui`'s `Shape::Callback`, который позволяет выполнить произвольный рендеринговый код внутри контекста `egui` [6]. Это даст больше контроля над отрисовкой и позволит добиться более точного имитации свечения сегментов. Для отображения специальных символов, таких как «EGGOG» или десятичной точки, потребуется аналогичная логика.

Симуляция клавиатуры также будет осуществляться через `egui`. В интерфейсе фронтенда необходимо будет создать сетку кнопок, имитирующую физическую клавиатуру MK-52. Каждая кнопка `egui` может быть связана с конкретным кодом клавиши. Когда пользователь нажимает кнопку на экране, фронтенд отправляет соответствующий код в бэкенд через канал сообщений. Бэкенд, в свою очередь, должен иметь механизм для регистрации нажатий. Поскольку сканирование клавиатуры происходит синхронно с таймером VFD, бэкенд должен в нужные моменты времени проверять наличие нажатий в очереди и обрабатывать их, как будто бы была нажата физическая клавиша. Это обеспечит корректную реакцию на ввод. Такой подход не только решает задачу симуляции, но и значительно упрощает отладку и использование эмулятора, поскольку пользователю не требуется никаких внешних устройств для ввода.

Важно отметить, что, хотя VFD требует высокого напряжения, его программная модель не нуждается в моделировании этих физических величин. Достаточно смоделировать его логическое поведение: какие сегменты горят в какой момент времени. Однако, для полной точности, стоит учесть, что VFD требует некоторого времени на прогрев и может гаснуть при длительных вычислениях, что влияет на энергопотребление [5] [11]. Энергопотребление в разных режимах (standby, вычисление) различается, что также может быть отражено в эмуляторе [11]. Таким образом, воспроизведение периферии — это задача не столько физического моделирования, сколько точного воссоздания логики и тайминга

ее работы, что полностью вписывается в парадигму цикл-точной эмуляции и является важным шагом к созданию качественного учебного прототипа.

Специфические Особенности и Недокументированные Функции: Путь к Полному Пониманию

Образовательная ценность эмулятора Электроника МК-52 будет определяться не только его способностью воспроизводить официально документированные функции, но и его точностью в отношении уникальных, а иногда и нежелательных особенностей, которые стали неотъемлемой частью архитектуры этих советских калькуляторов. Изучение этих "темных сторон" и недокументированных функций позволяет глубже понять ограничения и творческие возможности, заложенные разработчиками в аппаратное обеспечение. Для эмулятора, целью которого является полное воспроизведение оригинального поведения, корректная реализация этих состояний является обязательной. Анализ предоставленных источников выявляет несколько ключевых примеров таких состояний, включая ошибку "EGGOG", состояние "Тьма" и баг "пустышка", а также уникальные способы ввода данных, такие как работа с шестнадцатеричными числами.

Одной из самых известных и часто встречающихся характеристик МК-52 является сообщение об ошибке "EGGOG" [1] [3]. Это не просто случайная ошибка, а специально подобранное слово из отображаемых на VFD символов, которое использует меньше сегментов, чем прямой перевод слова "error" на кириллицу [1]. Сообщение "EGGOG" может появляться по разным причинам, включая переполнение при вычислениях (например, возведение в квадрат очень большого числа, результат которого выходит за пределы диапазона экспоненты) [3]. Критически важно, что появление "EGGOG" не всегда означает полный сбой системы. В некоторых случаях, если "EGGOG" был вызван переполнением, сама underlying числовая величина может быть сохранена в регистре и продолжать использоваться в дальнейших вычислениях [3]. Это открывает возможность для создания специфических программных состояний, которые могут быть использованы для входа в режим программирования или для запуска специальных алгоритмов. Эмулятор должен не только отображать "EGGOG", но и точно воспроизводить условия, при которых оно появляется, и

сохранять внутреннее значение числа, даже если оно не может быть отображено. Это требует точного моделирования диапазонов чисел и механизмов обработки переполнений в арифметическом процессоре.

Еще более экзотическим состоянием является "Тьма" («Тьма») [3]. Это состояние, которое можно вызвать, выполняя определенную последовательность арифметических операций на калькуляторе, уже находящемся в состоянии "EGGOG". В результате этого процесса экран полностью гаснет, и калькулятор становится полностью нечувствительным к нажатиям клавиш, за исключением сброса питания [3]. "Тьма" является прекрасным примером того, как неочевидные временные зависимости и побочные эффекты в работе аппаратных компонентов (VFD и процессоров) могут приводить к полному отказу системы. Для эмулятора, стремящегося к максимальной точности, важно не просто имитировать гашение экрана, но и воспроизвести последовательность действий, которая приводит к этому состоянию. Это требует от эмулятора не просто знать, как выглядит "Тьма", а понимать, какие внутренние изменения состояния (например, в регистре флагов или в состоянии VFD-контроллера) запускают этот переход. Только цикл-точная эмуляция, отслеживающая каждый шаг, может позволить воспроизвести такое сложное и чувствительное к времени состояние.

Наиболее любопытным аппаратным "багом" является "пустышка" [20]. Это состояние возникает, когда в памяти калькулятора (в программах или данных) случайно или намеренно появляется уникальный 9-полубитный паттерн, состоящий из одних единиц. Этот паттерн является маркером синхронизации внутри системы. Когда контроллер встречает его, он сбрасывает свой внутренний счетчик, что приводит к нарушению синхронизации с остальной частью системы. В результате система начинает интерпретировать данные как команды, а команды — как данные, что неминуемо ведет к полной коррупции памяти и невозможности дальнейшей работы устройства без перезагрузки [20]. Воспроизведение "пустышки" в эмуляторе — это сложная, но крайне полезная задача. Она требует от эмулятора точно моделировать не только хранение данных в круговом буфере, но и процесс их последовательного чтения и интерпретации процессором, шаг за шагом. Только эмулятор, который "прогоняет" каждый полубайт данных через свою модель памяти, сможет зафиксировать появление этого критического паттерна и корректно воспроизвести его разрушительные последствия. Это состояние является ярким примером того, как низкоуровневые детали аппаратной реализации могут влиять на всю систему.

Помимо состояний ошибок и багов, МК-52 обладает рядом уникальных неофициальных возможностей ввода данных. Особенно примечателен способ работы с шестнадцатеричными числами. Калькулятор не имеет специальных клавиш для шестнадцатеричных цифр A-F. Вместо этого они вводятся с помощью буквенных клавиш, которые на клавиатуре помечены кириллическими буквами [3]. Соответствие между шестнадцатеричными цифрами и клавишами выглядит следующим образом: A вводится нажатием клавиши '0', B — 'L', C — 'C', D — 'Г', E — 'E', а F не имеет клавиши [3]. При попытке ввести число, содержащее недопустимые цифры (например, EDD0D), результатом может стать отображение строки "Err0r", так как буква 'r' соответствует шестнадцатеричной цифре 6 [12]. Эмулятор должен точно воспроизводить это сопоставление клавиш, чтобы обеспечить совместимость с программами, написанными для оригинального устройства. Более того, существуют сложные техники для хранения текстовых строк в регистрах калькулятора, используя комбинации арифметических и логических функций, что свидетельствует о высокой степени гибкости архитектуры [12]. Например, для хранения строки "ERROR" в регистре 0 можно выполнить последовательность операций, используя функции логического ИЛИ (K) и извлечения дробной части ({x}) [12]. Корректная реализация таких хитростей является вершиной точности эмуляции и демонстрирует глубокое понимание внутренней логики калькулятора.

Включение в эмулятора всех этих неофициальных функций и состояний превращает его из простого инструмента в мощное исследовательское средство. Оно позволяет не только наблюдать за работой калькулятора, но и целенаправленно экспериментировать с его границами, воспроизводя исторические "открытия" и понимая причины возникновения этих явлений. Это полностью соответствует образовательной цели проекта — изучению архитектуры микроконтроллеров через полное воспроизведение их поведения. Для реализации этого в Rust-проекте необходимо, чтобы модель эмулятора была достаточно низкоуровневой, чтобы отслеживать внутренние состояния (например, содержимое регистров, состояние флагов, текущий тактовый цикл), а не просто внешние эффекты. Источники, такие как `mk-pro`, который использует JavaScript-эмулятор в качестве авторитетной модели для проверки оптимизаций, подчеркивают важность учета таких "темных входов" [32]. Таким образом, работа над этими специфическими особенностями должна стать центральным элементом процесса разработки и тестирования эмулятора.

Архитектура Проекта и Рекомендации по Реализации

Для успешной реализации амбициозной цели по созданию цикло-точного эмулятора Электроника МК-52 на Rust, необходимо заранее спроектировать четкую и масштабируемую архитектуру проекта. Анализ уникальной многопроцессорной архитектуры МК-52, требований к точности эмуляции и наличия передовых ресурсов позволяет сформулировать ряд рекомендаций, которые станут основой для разработки. Ключевые принципы, которые должны лежать в основе проекта: разделение на бэкенд и фронтенд, модульная структура бэкенда, использование существующих точных реализаций в качестве основы и фокус на цикл-точной парадигме с самого начала.

Первым и самым важным архитектурным решением является четкое разделение на бэкенд (логика эмуляции) и фронтенд (пользовательский интерфейс). Эта модульная структура является общепринятой практикой в разработке эмуляторов и обеспечивает гибкость, поддерживаемость и чистоту кода [16]. Бэкенд должен представлять собой самостоятельную библиотеку Rust (`lib.rs`), содержащую всю сложную логику эмуляции. Он не должен зависеть от какой-либо конкретной графической библиотеки или операционной системы. Внутри бэкенда должны быть созданы отдельные модули для каждого аппаратного компонента: `cpu_main`, `cpu_math`, `cpu_aux`, `memory_controller`, `vfd_controller`, `keyboard_scanner`, `eeprom_controller` и, возможно, `ring_buffer`. Каждый из этих модулей будет представлять собой объект, который в цикле эмуляции будет вызывать метод `tick()`, моделируя один тактовый цикл своей работы. Взаимодействие между этими модулями должно осуществляться через общий объект `MachineState`, который будет хранить состояние всей системы: содержимое кругового буфера, регистры, стек, флаги и т.д. Этот подход, подобный архитектуре эмулятора `ym2149-rs`, где каждая часть эмуляции зависит только от слоев ниже, обеспечивает высокую степень переиспользования и ясность [13].

Вторым ключевым принципом является стратегия разработки, основанная на реинжиниринге проекта `emu145`. Начинать разработку с нуля, пытаясь самостоятельно декодировать микрокод, было бы крайне трудоемко и подвержено ошибкам. Гораздо более эффективным подходом является использование существующего C++ кода `emu145` как основы [28] [29]. Процесс должен начаться с тщательного изучения этого кода, понимания его структуры

и логики. Затем следует создать в Rust-проекте аналогичную архитектуру, постепенно переводя логику из C++ в Rust. Этот процесс позволит сразу получить доступ к проверенной годами точной модели работы микроконтроллеров, что является краеугольным камнем всего проекта. При переводе необходимо уделять особое внимание адаптации кода к цикл-точной парадигме, которая, по-видимому, уже заложена в `emu145`.

Третий принцип — это фокус на цикл-точной эмуляции с самого первого дня разработки. Это означает, что на ранних этапах следует сосредоточиться на создании бэкенда, который точно моделирует взаимодействие процессоров и работу внутреннего таймера. Логика должна быть спроектирована так, чтобы каждый шаг соответствовал одному тактовому циклу (~75 кГц) или одной микрооперации [20]. Использование Rust-корутин (если допустимо) может значительно упростить реализацию этого шагового процесса [17]. Только после того, как будет создана работающая цикл-точная модель ядра системы, можно приступать к разработке фронтенда. Это позволит избежать ситуаций, когда фронтенд строится вокруг неточной бэкенд-логики.

Четвертый принцип — это использование `eframe` и `egui` для фронтенда. `egui` отлично подходит для создания быстрых и простых пользовательских интерфейсов для эмуляторов благодаря своей парадигме немедленного режима [6]. Фронтенд будет представлять собой обычное десктопное приложение, которое будет: 1. Создавать окно с помощью `eframe`. 2. В основном цикле рендерить интерфейс `egui`. 3. В интерфейсе `egui` размещать виртуальную клавиатуру MK-52 и большое поле для симуляции VFD. 4. Собирать события ввода (нажатия клавиш) и отправлять их в бэкенд через канал сообщений (`std::sync::mpsc::channel`) [6]. 5. Получать от бэкенда обновленное состояние VFD и отрисовывать его в `egui`. Для синхронизации потоков между бэкендом и фронтендом можно использовать `Arc<Mutex<...>>` для общих данных или каналы для асинхронной передачи событий и обновлений. Это обеспечит плавную работу интерфейса даже при высокой нагрузке на эмулятор.

Наконец, пятый принцип — это разработка плана тестирования, основывающегося на реальных программах. После создания базовой функциональности эмулятора, необходимо протестировать его на большом количестве программ, написанных для оригинального MK-52. Наличие коллекций программ, включающих игры, такие как "Лунолет-2", и математические утилиты, предоставляет готовый набор тестов для проверки

точности эмуляции [22] [31]. Особое внимание следует уделить тестированию на воспроизведение недокументированных состояний, таких как "EGGOG", "Тьма" и "пустышка" [3] [20]. Успешное воспроизведение этих состояний будет лучшим подтверждением высокой точности эмуляции и достижения образовательной цели проекта.

В таблице ниже представлена сводка рекомендуемой архитектуры проекта.

Компонент	Технология/ Подход	Описание	Ключевые функции
Бэкенд	Rust `lib`	Модульная библиотека, содержащая всю логику эмуляции.	`tick()` для каждого компонента, управление состоянием кругового буфера.
Архитектура	Реинжиниринг `emu145`	Основа проекта — точная реализация микрокода из C++ `emu145`.	Трансляция C++ логики на Rust, адаптация под цикл-точную модель.
Эмуляция	Цикл-точная	Каждый тактовый цикл (~75 кГц) моделируется последовательно.	Использование корутин (неустойчиво) или явное управление состоянием для шагового выполнения.
Модули	Модульная структура	Отдельные модули для каждого компонента (процессоры, контроллеры).	`cpu_main`, `vfd_controller`, `keyboard_scanner`, `eeprom_controller`.
Фронтенд	`eframe` + `egui`	Настольное приложение для визуализации и ввода.	Виртуальная клавиатура, отрисовка VFD, сбор ввода, отображение состояния.
Синхронизация	Каналы сообщений (`mpsc`)	Асинхронный обмен данными между бэкендом и фронтендом.	Передача событий клавиатуры в бэкенд и состояния VFD в фронтенд.
Тестирование	Реальные программы	Использование коллекций программ для проверки точности.	Тестирование на "Лунолете-2", крестиках-ноликах, программах для астронавтов.

Следуя этим рекомендациям, можно построить прочный фундамент для создания высокоэффективного эмулятора. Проект будет не просто рабочим, но и образовательно ценным инструментом, который точно воспроизводит уникальную архитектуру Электроника МК-52, позволяя глубоко изучить принципы работы микроконтроллеров через практическое применение.

Справка

1. Elektronika MK-52 https://en.wikipedia.org/wiki/Elektronika_MK-52

2. Elektronika MK 52 - Kees van der Sanden https://www.keesvandersanden.nl/calculators/elektronika_mk52.php
3. Mighty MK-52: The Spaceworthy Soviet-era Calculator https://electronicsdeli.net/2021/06/23/mk52/
4. Электроника МК-52 - Soviet calculator in orbit - Rafal Bartoszak https://rafalbartoszak.com/%D1%8D%D0%BB%D0%B5%D0%BA%D1%82%D1%80%D0%BE%D0%BD%D0%B8%D0%BA%D0%B0-%D0%BC%D0%BA-52-soviet-calculator-in-orbit/
5. Making Elektronika MK-52 VFD scanner http://achilikin.blogspot.com/2021/05/making-elektronika-mk-52-vfd-scanner.html
6. egui: an easy-to-use immediate mode GUI in Rust that runs ... https://github.com/emilk/egui
7. Egui – An experimental immediate mode GUI written in Rust https://lobste.rs/s/4wnemk/egui_experimental_immediate_mode_gui
8. What exactly is a cycle-accurate emulator? https://retrocomputing.stackexchange.com/questions/1191/what-exactly-is-a-cycle-accurate-emulator
9. Cycle-accurate NES emulator in around 1000 lines of code https://news.ycombinator.com/item?id=13052487
10. Электроника МК-52. Часть 1. Восстановление https://habr.com/ru/articles/710622/
11. Электроника МК-52 https://ru.wikipedia.org/wiki/%D0%AD%D0%BB%D0%B5%D0%BA%D1%82%D1%80%D0%BE%D0%BD%D0%B8%D0%BA%D0%B0_%D0%9C%D0%9A-52
12. Elektronika MK-52 Vintage Soviet Calculator - Linuxslate.com https://linuxslate.com/forum/viewtopic.php?t=27
13. YM2149 in Rust, Part 1: Building a Cycle-Accurate Emulator https://dev.to/markus_velten_e7807418293/ym2149-in-rust-part-1-building-a-cycle-accurate-emulator-33om
14. cycle-accurate NES emulator library in Rust (no_std, FFI, ... https://www.reddit.com/r/rust/comments/1uyune4/xnes_v01_cycleaccurate_nes_emulator_library_in/
15. AdamBlance/nes-emu: A cycle-accurate NES emulator ... https://github.com/AdamBlance/nes-emu
16. CHIP-8 emulator in Rust. Part 1 https://dhole.github.io/post/chip8_emu_1/
17. Cycle-accurate 6502 emulator as coroutine in Rust https://github.com/bagnalla/6502
18. Category:МК-61/52 https://rosettacode.org/wiki/Category:%D0%9C%D0%9A-61/52
19. Электроника МК-52 - Блог айтишника https://walhi.ru/archives/119

20. Внедрение в архитектуру советских программируемых ... https://habr.com/ru/articles/467501/
21. Вот эмулятор Intel 8080 с точным подсчетом циклов ... https://www.reddit.com/r/C_Programming/comments/8loz4p/heres_a_cycleaccurate_intel_8080_emulator_i_wrote/?tl=ru
22. Extended emulator of МК 61/54 https://f-droid.org/en/packages/com.cax.pmk.ext/
23. Reverse-engineering an early calculator chip with four- ... http://www.righto.com/2020/12/reverse-engineering-early-calculator.html
24. xvadim/pmk-android: Emulator of "Electronika MK 61" and " ... https://github.com/xvadim/pmk-android
25. "Электроника МК-52" с контроллером ESP32 http://arduino.ru/forum/proekty/elektronika-mk-52-s-kontrollerom-esp32
26. Расширенный эмулятор МК 61/54 https://f-droid.org/ru/packages/com.cax.pmk.ext/
27. Elektronika MK-52 ROM Functions Discussion | PDF https://www.scribd.com/document/707053760/Elektronika-MK-52-The-MK-61-big-brother-2
28. GitHub - BigEd/emu145: Automatically exported from code. ... https://github.com/BigEd/emu145
29. Программируемый микрокалькулятор МК-61 https://sergeanvarov.github.io/russian/mk61/index.html
30. эмулятор и симулятор советских калькуляторов http://www.emulator3000.org/rus-c3.htm
31. Электроника МК 61 - Игровая система. Лунолет-2 ... https://www.youtube.com/watch?v=riyjCp97mEY
32. bolknote/MKPro: High-level MK-Pro compiler ... https://github.com/bolknote/MKPro
33. Felix Lazarev fixelsan https://github.com/fixelsan